IA-graphie — usage des IA génératives

SAÉ 3.02 — Groupe « Python Mayo »

Atakan Gocer · Louise Halter-Tisler

Ce document recense l'usage que nous avons fait des outils d'IA générative pendant le projet, les prompts qui ont réellement structuré notre travail, et notre analyse critique de ce qu'ils nous ont apporté ou coûté.

Outil utilisé : Claude (Anthropic), d'abord par l'interface web pour la conception, puis via Claude Code pour le développement et le déploiement.

---

1. Cartographie de l'usage

Vue d'ensemble

Phase	Période	Usage de l'IA	Notre part
Analyse du besoin	septembre	Reformulation de la fiche QQOQCP, liste de questions à poser au client	Choix des questions retenues, entretien avec le client
Architecture initiale	septembre	Proposition de découpage en modules et de schéma de base	Validation, puis correction après lecture du cahier des charges
Premier prototype	septembre	Génération du squelette de code	Tests, repérage des écarts avec l'attendu
Changement de pile	septembre	Réécriture de la couche technique vers PyQt6 et sockets	Décision du changement, imposée par le cahier des charges
Logique métier	septembre – octobre	Implémentation des règles de circulation et de priorité	Spécification des règles, validation du comportement
Analyse critique	octobre	Auto-critique du système et proposition d'une étude par ablation	Arbitrage sur ce qui méritait d'être corrigé
Répartition sur deux VM	novembre	Découpage en deux processus, scripts de déploiement	Contrainte posée par le client, validation par le test
Déploiement et débogage	novembre	Diagnostic des pannes réseau, correction des scripts	**Tous les tests sur VM, qui ont révélé les bugs**
Documentation	novembre	Rédaction des guides et de la documentation technique	Relecture, signalement des étapes manquantes

Ce que l'IA a fait

- **Produire du code à partir de spécifications** que nous donnions en français : règles de priorité, cycle des feux, protocole réseau.
- **Expliquer des notions** que nous ne maîtrisions pas : la différence entre un flux TCP et un message, la règle de thread-safety de Qt, le fonctionnement des profils NetworkManager.
- **Diagnostiquer des pannes** à partir des messages d'erreur et des captures d'écran que nous lui transmettions.
- **Rédiger la documentation** à partir du code existant et des problèmes rencontrés pendant nos tests.

Ce que l'IA n'a pas fait

- **Décider du périmètre.** Le choix de traiter la voie réservée plutôt qu'une simple préemption des feux vient de notre lecture du besoin.
- **Tester.** Aucun test sur les VM n'a été fait par l'IA : elle n'y avait pas accès. Tous les déploiements, et donc toutes les découvertes de bugs, viennent de nos manipulations.
- **Arbitrer les risques.** La décision de ne pas refactorer le code à deux jours du rendu, par exemple, a été prise après discussion.

---

2. Traçabilité des prompts structurants

Nous retenons ici les demandes qui ont réellement infléchi le projet, avec notre analyse de la réponse obtenue.

Prompt 1 — Le cadrage initial

> « je souhaite faire ce projet tout est expliqué dans ce document fais moi un récapitulatif de ce que tu comprends »

Réponse obtenue : une reformulation structurée de la fiche QQOQCP, suivie d'une liste de points flous à clarifier avec le client.

Analyse critique. Utile pour vérifier notre propre compréhension, mais la reformulation ne contenait rien que le document ne disait déjà. La vraie valeur était dans la liste des zones d'ombre, qui nous a servi à préparer l'entretien. En revanche, l'outil a enchaîné sur une proposition d'architecture **avant** que nous ayons les réponses du client : nous aurions dû attendre.

Prompt 2 — Le choix de la pile technique

> « je veux commencer le projet rédige moi le code avec un affichage graphique ect une base »

Réponse obtenue : un prototype complet en **Pygame** avec MariaDB.

Analyse critique. C'est notre plus grosse perte de temps du projet, et elle est de notre fait. La demande ne précisait aucune contrainte, alors que le cahier des charges imposait PyQt6 et interdisait explicitement Pygame. L'outil a proposé la solution la plus courante pour une simulation 2D, ce qui était raisonnable faute d'information. Le prototype a dû être entièrement réécrit quand nous avons transmis le cahier des charges.
Leçon retenue : fournir les contraintes avant de demander du code, pas après.

Prompt 3 — La demande d'auto-critique

> *(demande de regard critique sur le système produit)*

Réponse obtenue : une liste de faiblesses, dont le fait que le gain attribué à la voie réservée n'était pas isolé de celui du simple feu vert forcé.

Analyse critique. C'est le prompt le plus utile du projet. Il a produit une remise en cause que nous n'avions pas vue : nos chiffres ne prouvaient pas ce que nous affirmions. Il en est sorti l'étude comparative par ablation, qui a révélé le résultat le plus intéressant du projet — la préemption des feux seule est **pire** que ne rien faire. Sans cette demande, nous aurions présenté une conclusion fausse.

Prompt 4 — La répartition sur deux VM

> « cette répartition doit se faire à travers les processus [...] il ne voulait pas que tous les processus soient sur une VM, donc on doit répartir les processus sur les 2 VM, mais également les bases de données »

Réponse obtenue : une proposition d'architecture séparant le moteur de la passerelle, avec une base auprès de chaque processus, puis l'implémentation.

Analyse critique. La contrainte venait du client, pas de l'outil. Ce que l'IA a apporté, c'est la **justification** du découpage — colocaliser chaque base avec son principal producteur d'écritures — que nous avons reprise à notre compte parce qu'elle tient. Nous avons cependant dû choisir nous-mêmes entre plusieurs découpages possibles.

Prompt 5 — Le déploiement réel

> *(envoi de captures d'écran des erreurs rencontrées sur les VM)*

Réponse obtenue : des diagnostics successifs, dont certains faux avant d'être corrigés.

Analyse critique. C'est ici que la collaboration a le mieux fonctionné, parce que nous apportions quelque chose que l'outil n'avait pas : **l'accès aux machines**. Nos tests ont révélé des bugs que l'IA avait elle-même introduits et qu'aucune relecture n'aurait détectés. Nous y revenons au chapitre suivant.

---

3. Ce que nos tests ont révélé

Ce chapitre est le plus instructif du document, parce qu'il montre la limite concrète de l'outil : **du code plausible, relu et commenté, peut être faux**, et seule l'exécution le dit.

Un bug invisible en relecture

Le DAO lisait `usager.direction`, un attribut qui n'existe pas — il s'appelle `branche`. Le code était syntaxiquement correct, cohérent avec son contexte, et avait été relu.

Pourquoi personne ne l'a vu : **ce chemin n'était jamais exécuté**. Sans MariaDB installé en local, le DAO valait `None` et les écritures en base étaient silencieusement ignorées. Le bug n'est apparu qu'au premier déploiement sur une VM, où le moteur a planté dès le premier usager.

Un bug qui simulait son propre bon fonctionnement

Le script de provisionnement cherchait la carte réseau à configurer en listant celles qui **avaient déjà une adresse**. Or la carte privée n'en a pas, c'est précisément ce qu'on veut corriger. Il sélectionnait donc la carte Internet et lui imposait une adresse privée.

Résultat : deux routes concurrentes vers le même réseau, une machine invisible pour l'autre VM — alors que tous les signes extérieurs paraissaient bons. Interface active, bonne adresse, aucun message d'erreur. Il a fallu lire la table de routage pour comprendre.

Un message qui affirmait le contraire de la réalité

La passerelle affichait `Bilan de session enregistre` même lorsque rien n'avait été écrit en base. Nous avons cru pendant un moment que nos données étaient enregistrées.

Ce que nous en retenons

L'outil produit vite du code qui a toutes les apparences du correct. Les trois bugs ci-dessus ont en commun de ne pas être détectables par la lecture : il fallait **exécuter dans les conditions réelles**. Notre contribution décisive n'a pas été d'écrire ce code, mais de le mettre en situation jusqu'à ce qu'il casse.

C'est aussi ce qui nous a fait refuser une dernière proposition : à deux jours du rendu, simplifier le code pour le rendre plus facile à expliquer. Sans tests automatisés, toute modification aurait dû être revalidée à la main sur les deux VM. Nous avons préféré documenter le code existant plutôt que risquer un projet qui fonctionne.

---

4. Réflexion éthique

Propriété intellectuelle du code produit

Une part importante du code a été écrite par un outil entraîné sur du code existant, dont nous ne connaissons ni les sources ni les licences. Nous ne pouvons donc pas garantir l'originalité de chaque ligne. Dans un contexte scolaire, c'est acceptable dès lors que l'usage est déclaré — ce document existe pour cela. Dans un contexte professionnel, le sujet mériterait une vérification juridique avant diffusion.

Nous assumons en revanche la **responsabilité** de ce code : nous l'avons lu, testé, corrigé, et nous sommes capables de l'expliquer. Un code qu'on ne saurait pas défendre ne devrait pas être rendu.

Dépendance technique

Le risque est réel et nous l'avons ressenti. Il est confortable de demander une correction plutôt que de comprendre une panne. Deux garde-fous nous ont aidés : **exiger systématiquement l'explication** avec le correctif, et **tester nous-mêmes**, ce qui oblige à comprendre ce qu'on observe.

Le chapitre précédent montre d'ailleurs la limite de cette dépendance : l'outil ne pouvait pas trouver seul des bugs qui ne se manifestent qu'à l'exécution. Lui déléguer aussi la validation aurait laissé passer les trois.

Données personnelles

Le projet ne manipule aucune donnée personnelle réelle : les usagers sont générés aléatoirement et ne portent qu'un identifiant numérique, un type et des horodatages. Aucune question de conformité au RGPD ne se pose en l'état.

Elle se poserait immédiatement pour un déploiement réel à Mulhouse : un système qui détecte les véhicules et enregistre leurs passages traite des données susceptibles d'identifier des personnes. Il faudrait alors définir une durée de conservation, limiter la collecte au strict nécessaire, et probablement anonymiser les identifiants.

Biais et angles morts

L'outil propose les solutions les plus répandues dans son corpus. Cela s'est vu dès le début : interrogé sans contrainte, il a proposé Pygame, le choix le plus fréquent pour une simulation 2D en Python, alors que notre cahier des charges l'interdisait.

Le même biais s'applique aux choix de conception : les solutions proposées sont conventionnelles, rarement originales. Pour un projet scolaire, c'est plutôt un avantage — le code est lisible et reconnaissable. Pour un projet de recherche, ce serait une limite.

Honnêteté de la démarche

Nous avons choisi de documenter notre usage de l'IA avec précision, y compris ce qui ne nous met pas en valeur : le prototype Pygame à réécrire, les bugs introduits, le temps perdu. Un document qui ne présenterait que les réussites serait moins utile et moins crédible.